

深度学习

张烨菲

2025年11月

第七章 深度前馈神经网络

- 7.1 示例：学习 XOR
- 7.2 基于梯度的学习
- 7.3 隐藏单元
- 7.4 架构设计
- 7.5 反向传播

7 深度前馈神经网络

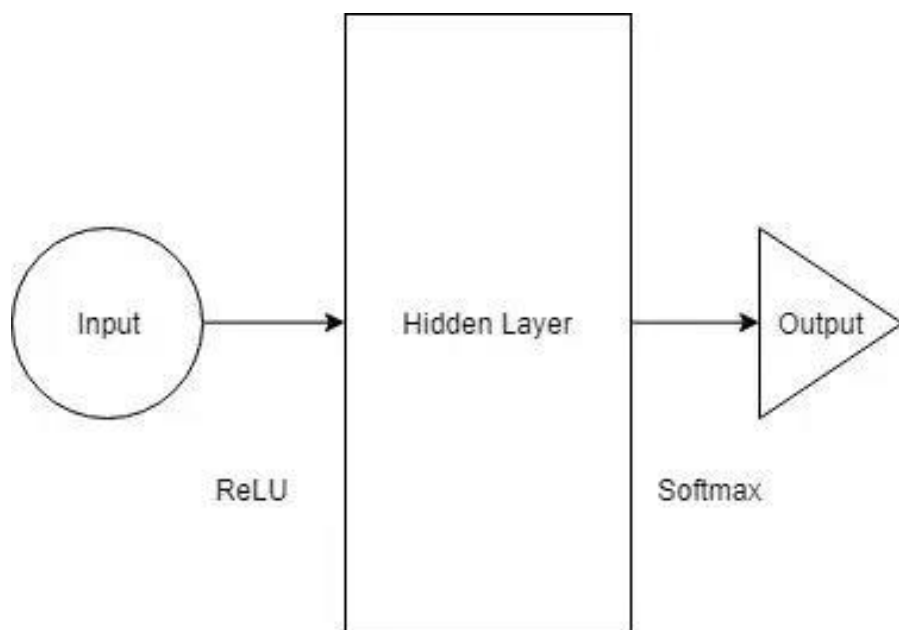
3

深度前馈网络 (deep feedforward network), 也叫作 前馈神经网络 (feedforward neural network) 或者 多层感知机 (multilayer perceptron, MLP), 是典型的深度学习模型。前馈网络的目标是近似某个函数 f^* 。例如, 对于分类器, $y = f^*(\mathbf{x})$ 将输入 $\mathbf{x}$ 映射到一个类别 y 。前馈网络定义了一个映射 $\mathbf{y} = f(\mathbf{x}; \boldsymbol{\theta})$, 并且学习参数 $\boldsymbol{\theta}$ 的值, 使它能够得到最佳的函数近似。

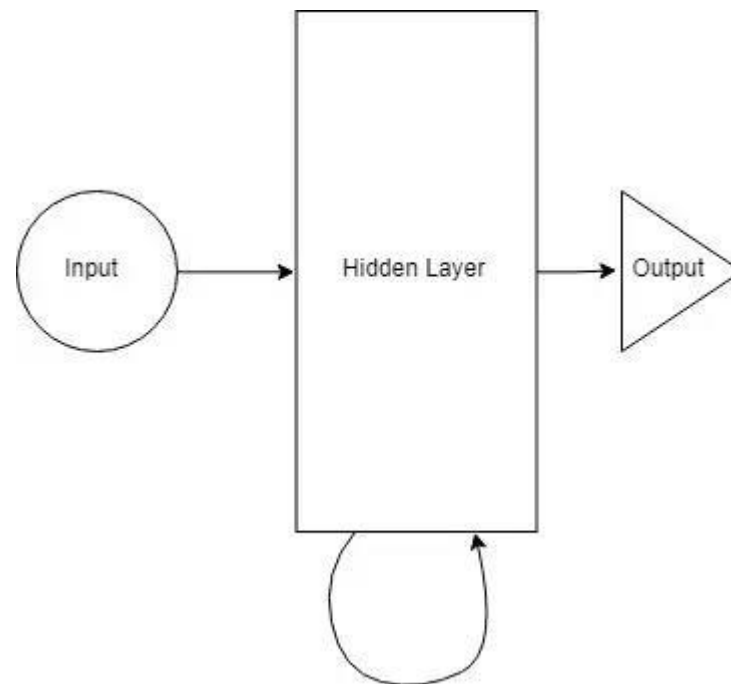
7 深度前馈神经网络

4

前馈神经网络



循环神经网络

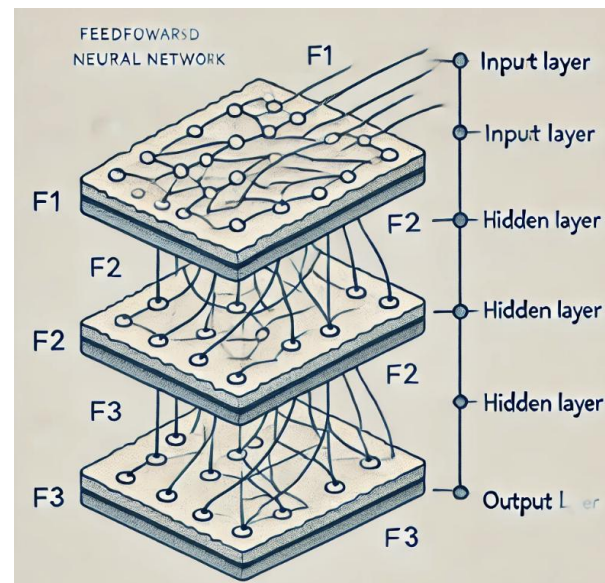


深度前馈神经网络在模型的输出和模型本身之间没有反馈（**feedback**）连接

7 深度前馈神经网络

5

前馈神经网络被称作网络（network）是因为它们通常用许多不同函数复合在一起来表示。该模型与一个有向无环图相关联，而图描述了函数是如何复合在一起的。例如，我们有三个函数 $f^{(1)}$, $f^{(2)}$ 和 $f^{(3)}$ 连接在一个链上以形成 $f(\mathbf{x}) = f^{(3)}(f^{(2)}(f^{(1)}(\mathbf{x})))$ 。这些链式结构是神经网络中最常用的结构。在这种情况下， $f^{(1)}$ 被称为网络的**第一层**（first layer）， $f^{(2)}$ 被称为**第二层**（second layer），以此类推。链的全长称为模型的**深度**（depth）。



7 深度前馈神经网络

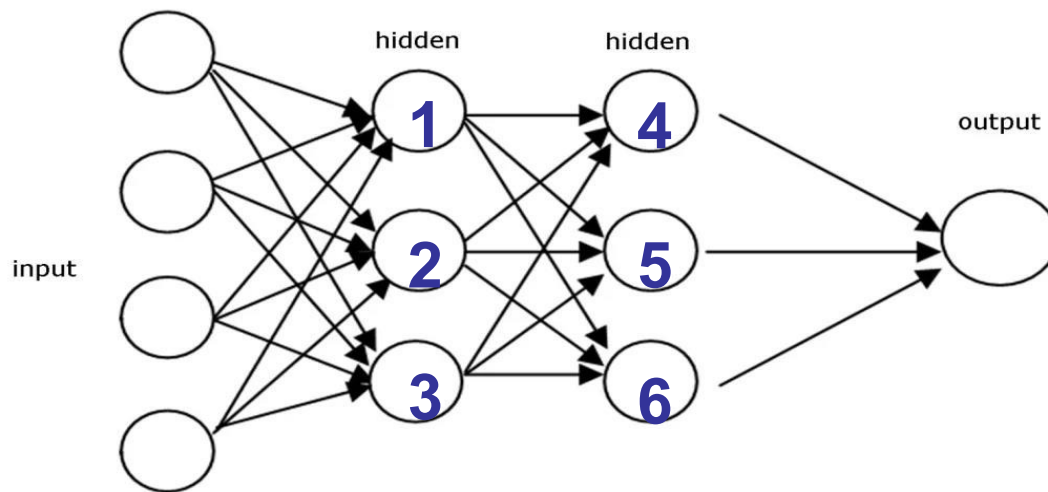
6

前馈网络的最后一层被称为**输出层** (output layer)。在神经网络训练的过程中，我们让 $f(\mathbf{x})$ 去匹配 $f^*(\mathbf{x})$ 的值。训练数据为我们提供了在不同训练点上取值的、含有噪声的 $f^*(\mathbf{x})$ 的近似实例。每个样本 $\mathbf{x}$ 都伴随着一个标签 $y \approx f^*(\mathbf{x})$ 。训练样本直接指明了输出层在每一点 $\mathbf{x}$ 上必须做什么；它必须产生一个接近 y 的值。但是训练数据并没有直接指明其他层应该怎么做。学习算法必须决定如何使用这些层来产生想要的输出，但是训练数据并没有说每个单独的层应该做什么。相反，学习算法必须决定如何使用这些层来最好地实现 f^* 的近似。因为训练数据并没有给出这些层中的每一层所需的输出，所以这些层被称为**隐藏层** (hidden layer)。

7 深度前馈神经网络

7

这些网络被称为神经网络是因为它们或多或少地受到神经科学的启发。网络中的每个隐藏层通常都是向量值的。这些隐藏层的维数决定了模型的宽度 (width)。向量的每个元素都可以被视为起到类似一个神经元的作用。除了将层想象成向量到向量的单个函数，我们也可以把层想象成由许多并行操作的**单元 (unit)**组成，每个单元表示一个向量到标量的函数。每个单元在某种意义上类似一个神经元，它接收的输入来源于许多其他的单元，并计算它自己的激活值。使用多层向量值表示的想法来源于神经科学。

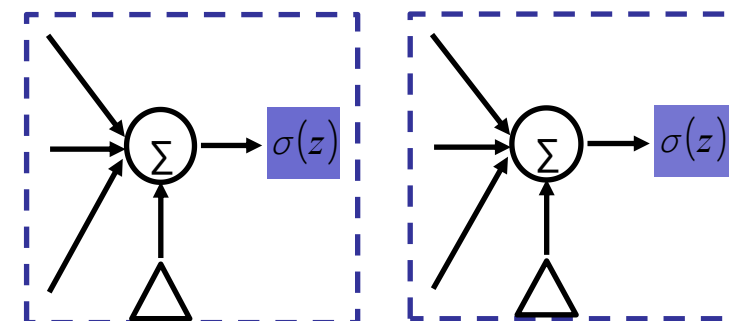
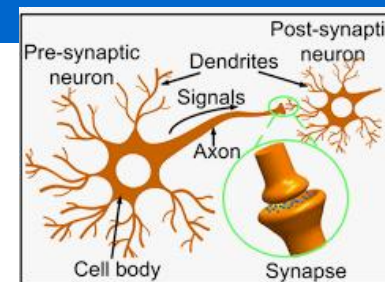
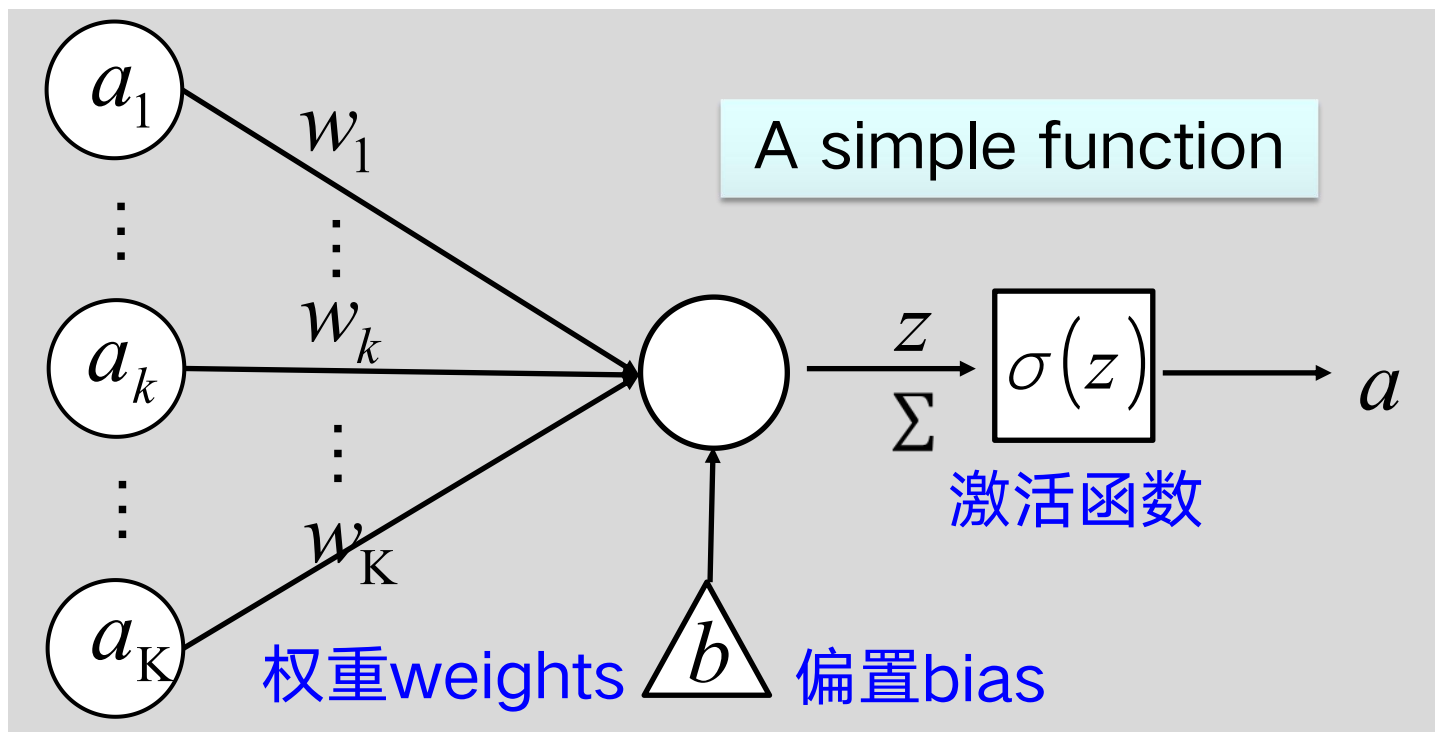


7 深度前馈神经网络

8

神经元是深度神经网络中最基础的组成单元。

$$z = a_1 w_1 + \cdots + a_k w_k + \cdots + a_K w_K + b$$



每个神经元都有自己的
权重和偏置参数

7 深度前馈神经网络

9

部署一个前馈网络：需要选择一个优化模型、代价函数以及输出单元的形式。

- 设计网络的结构，包括网络应该包含多少层、这些层应该如何连接，以及每一层包含多少单元。
- 前馈网络已经引入了隐藏层的概念，这需要我们选择用于计算隐藏层值的激活函数（activation function）。
- 在深度神经网络的学习中需要计算复杂函数的梯度，利用反向传播（back propagation）算法和它的现代推广可以用来高效地计算这些梯度。

7.1 示例：学习 XOR

10

XOR 函数（“异或”逻辑）是两个二进制值 x_1 和 x_2 的运算。当这些二进制值中恰好有一个为 1 时，XOR 函数返回值为 1。其余情况下返回值为 0。XOR 函数提供了我们想要学习的目标函数 $y = f^*(\mathbf{x})$ 。我们的模型给出了一个函数 $y = f(\mathbf{x}; \boldsymbol{\theta})$ 并且我们的学习算法会不断调整参数 $\boldsymbol{\theta}$ 来使得 f 尽可能接近 f^* 。

7.1 示例：学习 XOR

11

在这个简单的例子中，我们不会关心统计泛化。我们希望网络在这四个点 $\mathbb{X} = \{[0, 0]^\top, [0, 1]^\top, [1, 0]^\top, [1, 1]^\top\}$ 上表现正确。我们会用全部这四个点来训练我们的网络，唯一的挑战是拟合训练集。

A	B	输出Y
0	0	0
0	1	1
1	0	1
1	1	0

$$J(\boldsymbol{\theta}) = \frac{1}{4} \sum_{\mathbf{x} \in \mathbb{X}} (f^*(\mathbf{x}) - f(\mathbf{x}; \boldsymbol{\theta}))^2$$

7.1 示例：学习 XOR

12

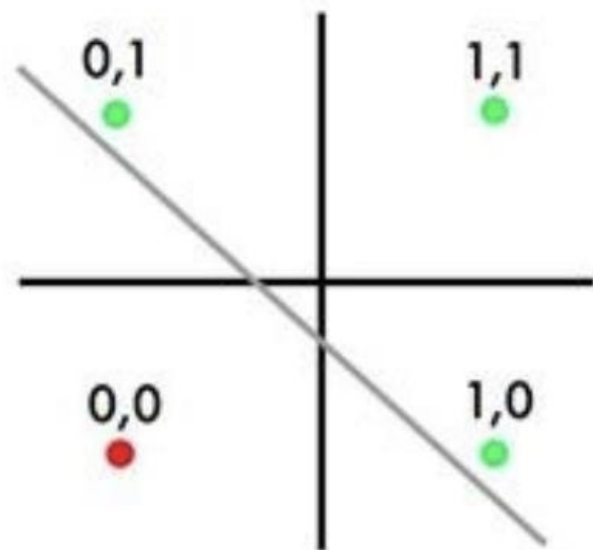
我们现在必须要选择我们模型 $f(\mathbf{x}; \boldsymbol{\theta})$ 的形式。假设我们选择一个线性模型， $\boldsymbol{\theta}$ 包含 $\mathbf{w}$ 和 b ，那么我们的模型被定义成

$$f(\mathbf{x}; \mathbf{w}, b) = \mathbf{x}^\top \mathbf{w} + b. \quad (6.2)$$

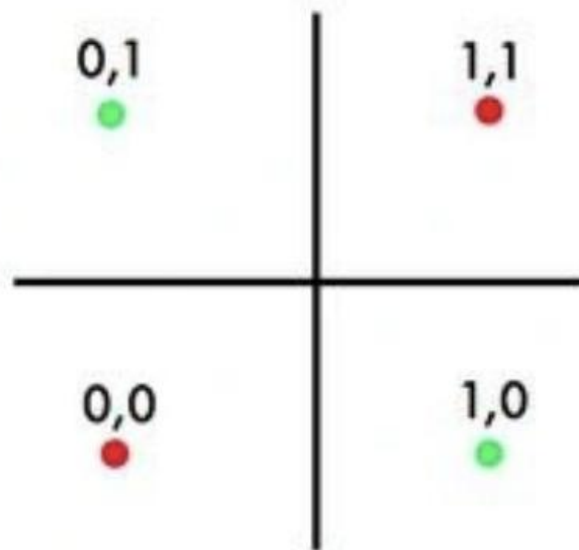
我们可以使用正规方程关于 $\mathbf{w}$ 和 b 最小化 $J(\boldsymbol{\theta})$ ，来得到一个闭式解。

7.1 示例：学习 XOR

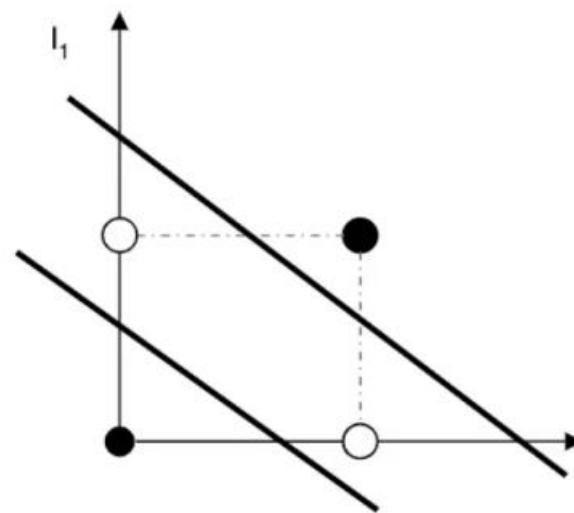
13



OR

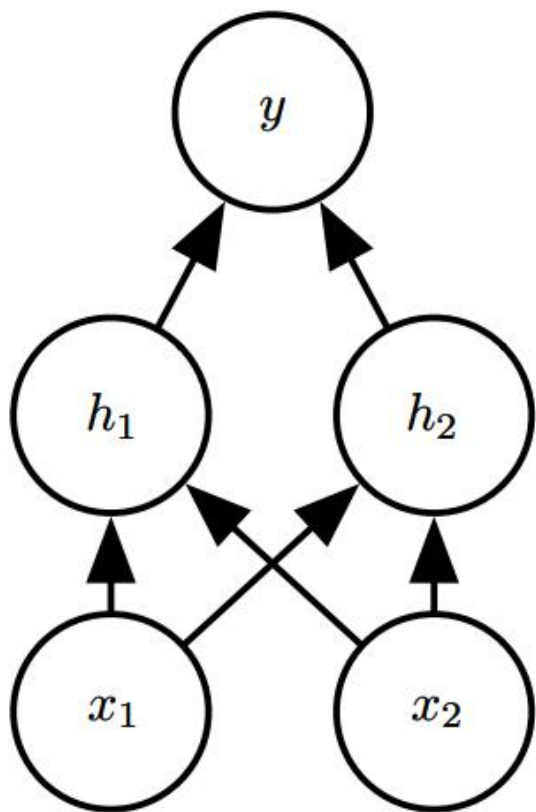


XOR



7.1 示例：学习 XOR

14



$$f(\mathbf{x}; \mathbf{W}, \mathbf{c}, \mathbf{w}, b) = f^{(2)}(f^{(1)}(\mathbf{x}))$$

$$\mathbf{h} = f^{(1)}(\mathbf{x}; \mathbf{W}, \mathbf{c})$$

$$y = f^{(2)}(\mathbf{h}; \mathbf{w}, b)$$

7.1 示例：学习 XOR

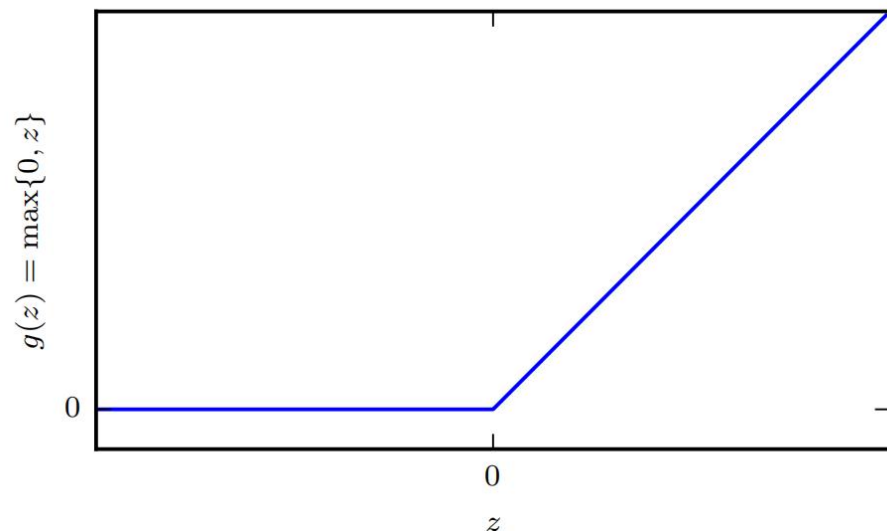
15

$f^{(1)}$ 应该是哪种函数？线性模型到目前为止都表现不错，让 $f^{(1)}$ 也是线性的似乎很有诱惑力。可惜的是，如果 $f^{(1)}$ 是线性的，那么前馈网络作为一个整体对于输入仍然是线性的。暂时忽略截距项，假设 $f^{(1)}(\mathbf{x}) = \mathbf{W}^\top \mathbf{x}$ 并且 $f^{(2)}(\mathbf{h}) = \mathbf{h}^\top \mathbf{w}$ ，那么 $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{W}^\top \mathbf{x}$ 。我们可以将这个函数重新表示成 $f(\mathbf{x}) = \mathbf{x}^\top \mathbf{w}'$ 其中 $\mathbf{w}' = \mathbf{W}\mathbf{w}$ 。

7.1 示例：学习 XOR

16

大多数神经网络通过仿射变换之后紧跟着一个被称为激活函数的固定非线性函数来实现这个目标，其中仿射变换由学得的控制。



$$\mathbf{h} = g(\mathbf{W}^\top \mathbf{x} + \mathbf{c})$$

$$h_i = g(\mathbf{x}^\top \mathbf{W}_{:,i} + c_i)$$

$$g(z) = \max\{0, z\} \Rightarrow \text{ReLU}$$

7.1 示例：学习 XOR

17

$$f(\mathbf{x}; \mathbf{W}, \mathbf{c}, \mathbf{w}, b) = \mathbf{w}^\top \max\{0, \mathbf{W}^\top \mathbf{x} + \mathbf{c}\} + b.$$

$$\mathbf{W} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \quad \mathbf{c} = \begin{bmatrix} 0 \\ -1 \end{bmatrix}, \quad \mathbf{X} = \begin{bmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{bmatrix}.$$
$$\mathbf{w} = \begin{bmatrix} 1 \\ -2 \end{bmatrix}, \quad b = 0$$

7.1 示例：学习 XOR

18

输入样本 1: $x_1 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$

1. 计算 $W^\top x_1 + c$:

$$W^\top x_1 + c = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ -1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ -1 \end{bmatrix} = \begin{bmatrix} 0 \\ -1 \end{bmatrix}$$

2. 应用ReLU激活函数:

$$\max \{0, W^\top x_1 + c\} = \max \left\{ 0, \begin{bmatrix} 0 \\ -1 \end{bmatrix} \right\} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

3. 计算最终输出:

$$f(x_1) = w^\top \max \{0, W^\top x_1 + c\} + b = \begin{bmatrix} 1 \\ -2 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \end{bmatrix} + 0 = 0$$

[0]

输入样本 2: $x_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$

1. 计算 $W^\top x_2 + c$:

$$W^\top x_2 + c = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} + \begin{bmatrix} 0 \\ -1 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix} + \begin{bmatrix} 0 \\ -1 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

2. 应用ReLU激活函数:

$$\max \{0, W^\top x_2 + c\} = \max \left\{ 0, \begin{bmatrix} 1 \\ 0 \end{bmatrix} \right\} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

3. 计算最终输出:

$$f(x_2) = w^\top \max \{0, W^\top x_2 + c\} + b = \begin{bmatrix} 1 \\ -2 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} + 0 = 1$$

7.1 示例：学习 XOR

19

输入样本 3: $x_3 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$

1. 计算 $W^\top x_3 + c$:

$$W^\top x_3 + c = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ -1 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix} + \begin{bmatrix} 0 \\ -1 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

2. 应用ReLU激活函数:

$$\max \left\{ 0, W^\top x_3 + c \right\} = \max \left\{ 0, \begin{bmatrix} 1 \\ 0 \end{bmatrix} \right\} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

3. 计算最终输出:

$$f(x_3) = w^\top \max \left\{ 0, W^\top x_3 + c \right\} + b = \begin{bmatrix} 1 \\ -2 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} + 0 = 1$$

输入样本 4: $x_4 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$

1. 计算 $W^\top x_4 + c$:

$$W^\top x_4 + c = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} + \begin{bmatrix} 0 \\ -1 \end{bmatrix} = \begin{bmatrix} 2 \\ 2 \end{bmatrix} + \begin{bmatrix} 0 \\ -1 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

2. 应用ReLU激活函数:

$$\max \left\{ 0, W^\top x_4 + c \right\} = \max \left\{ 0, \begin{bmatrix} 2 \\ 1 \end{bmatrix} \right\} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

3. 计算最终输出:

$$f(x_4) = w^\top \max \left\{ 0, W^\top x_4 + c \right\} + b = \begin{bmatrix} 1 \\ -2 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix} + 0 = 2 - 2 = 0$$

7.1 示例：学习 XOR

20

$$\mathbf{X} = \begin{bmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{bmatrix}.$$

$$\mathbf{w}^\top \max\{0, \mathbf{W}^\top \mathbf{x} + \mathbf{c}\}$$

$$\mathbf{X}\mathbf{W} = \begin{bmatrix} 0 & 0 \\ 1 & 1 \\ 1 & 1 \\ 2 & 2 \end{bmatrix} \cdot \begin{bmatrix} 0 & -1 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{bmatrix} \cdot \begin{bmatrix} 0 & 0 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{bmatrix} \cdot \begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}$$

$$\mathbf{W}^\top \mathbf{x} + \mathbf{c} \quad \max\{0, \mathbf{W}^\top \mathbf{x} + \mathbf{c}\}$$

7.2 基于梯度的学习

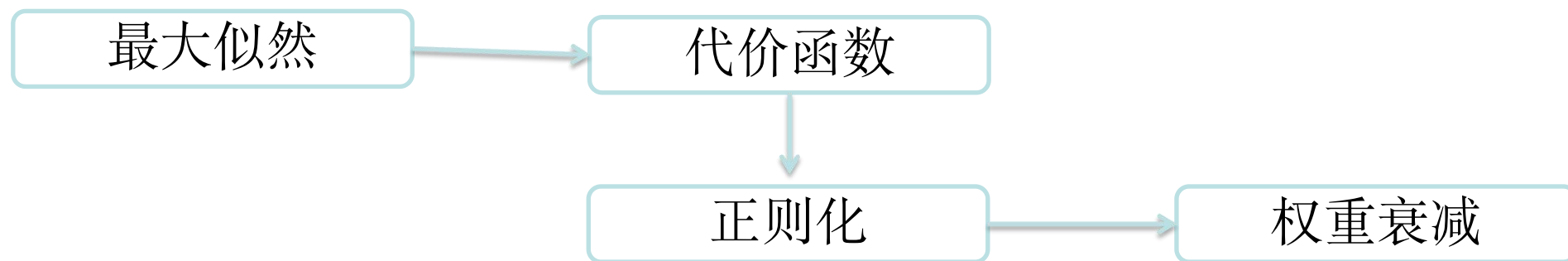
21

在这个例子中，我们简单地指定了解决方案，然后说明它得到的误差为零。在实际情况下，可能会有数十亿的模型参数以及数十亿的训练样本，所以不能像我们这里做的那样进行简单地猜解。与之相对的，基于梯度的优化算法可以找到一些参数使得产生的误差非常小。我们这里给出的 XOR 问题的解处在损失函数的全局最小点，所以梯度下降算法可以收敛到这一点。梯度下降算法还可以找到 XOR 问题一些其他的等价解。梯度下降算法的收敛点取决于参数的初始值。在实践中，梯度下降通常不会找到像我们这里给出的那种干净的、容易理解的、整数值的解。

7.2.1 代价函数

22

在大多数情况下，我们的参数模型定义了一个分布 $p(\mathbf{y} | \mathbf{x}; \boldsymbol{\theta})$ 并且我们简单地使用最大似然原理。这意味着我们使用训练数据和模型预测间的交叉熵作为代价函数。



7.2.1 代价函数

23

- 最大化观测数据的似然函数, 似然函数的对数就是负的交叉熵。
- 通过最小化交叉熵代价函数, 就相当于最大化似然函数, 也就是在寻找最有可能产生观测数据的参数值。

$$J(\boldsymbol{\theta}) = -\mathbb{E}_{\mathbf{x}, \mathbf{y} \sim \hat{p}_{\text{data}}} \log p_{\text{model}}(\mathbf{y} \mid \mathbf{x}).$$

- $\hat{p}_{\text{data}}$ 是训练数据的经验分布
- $p_{\text{model}}(\mathbf{y} \mid \mathbf{x})$ 是模型预测的条件概率

7.2.1 代价函数

24

使用最大似然来导出代价函数的方法的一个优势是，它减轻了为每个模型设计代价函数的负担。明确一个模型 $p(\mathbf{y} | \mathbf{x})$ 则自动地确定了一个代价函数 $\log p(\mathbf{y} | \mathbf{x})$ 。

假设分布	输出单元	代价函数	适用问题
高斯分布	线性单元	均方误差损失函数	回归
伯努利分布	Sigmiod	交叉熵损失函数	二分类
多项式分布	Softmax	多类别交叉熵损失函数	多分类

7.2.1 代价函数

25

面对以下问题，应当用什么输出单元，如何设计代价函数：

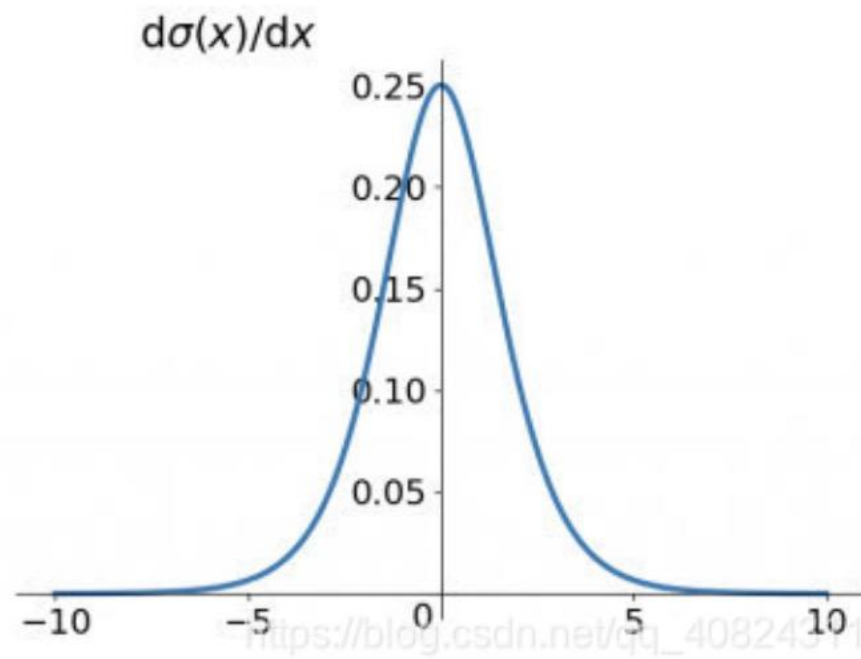
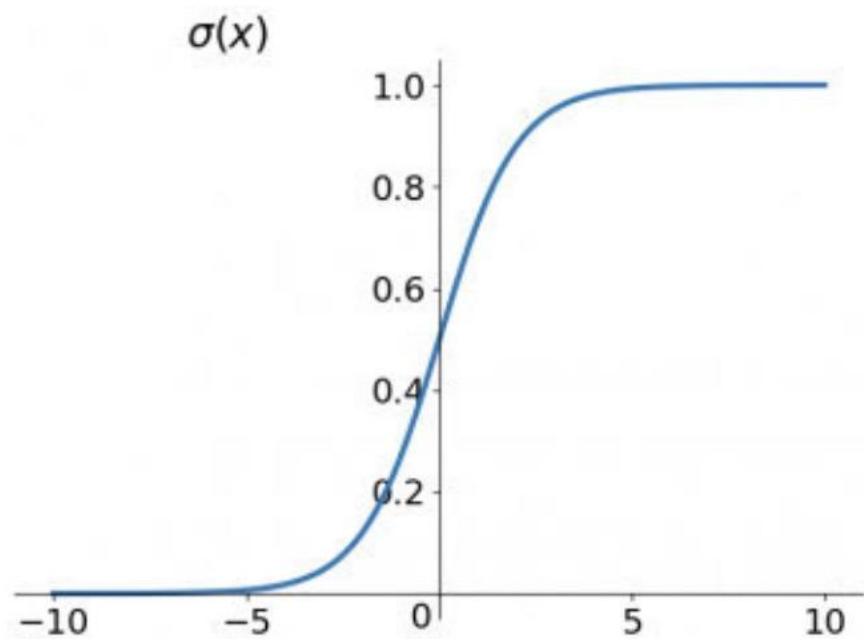
- 预测房价
- 预测气温
- 垃圾邮件分类
- 图像分类

7.2.1 代价函数

26

饱和、梯度消失

$\text{sigmoid}(x) = \frac{1}{1+e^{-x}}$ ，其导数为 $\text{sigmoid}(x) = \frac{e^{-x}}{(1+e^{-x})^2}$ ，图像如下：

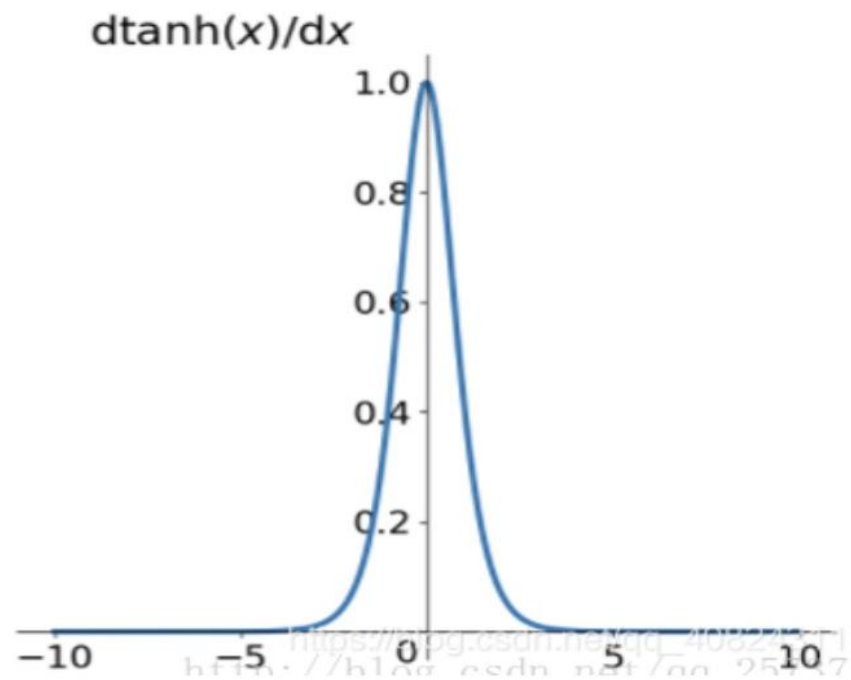
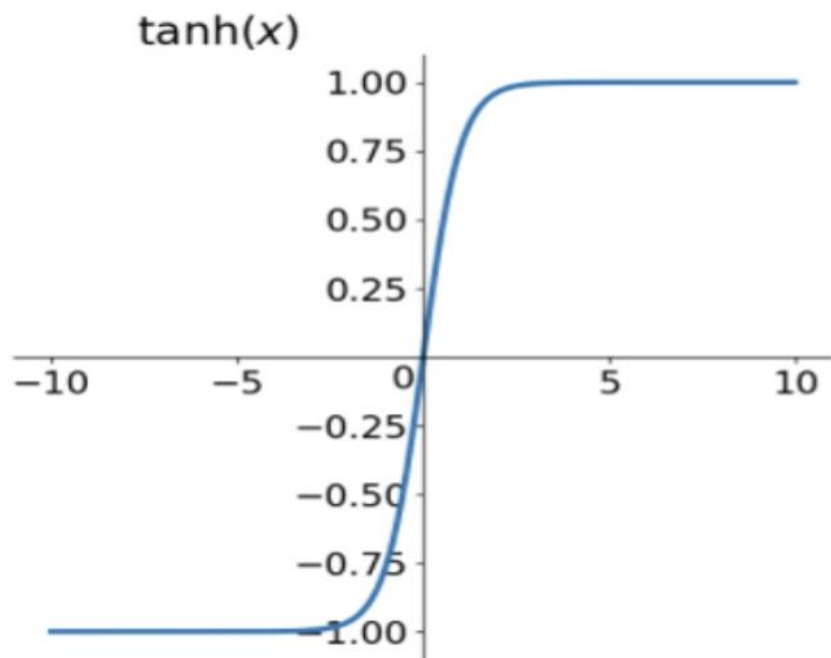


7.2.1 代价函数

27

饱和、梯度消失

$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$ ，其导数为 $\frac{4e^{2x}}{(e^{2x} + 1)^2}$ ，图像如下：

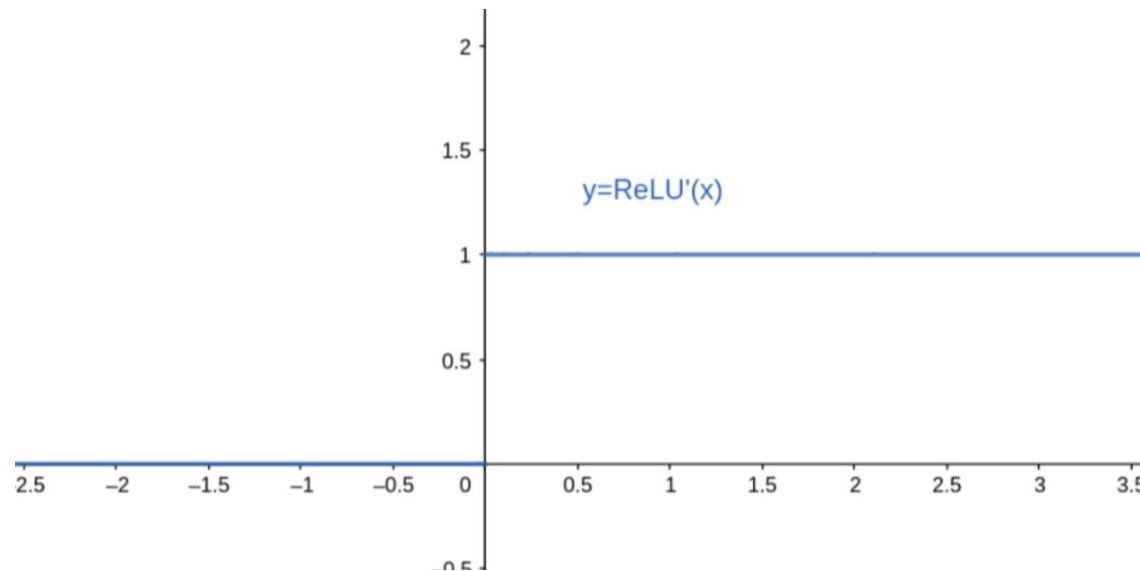
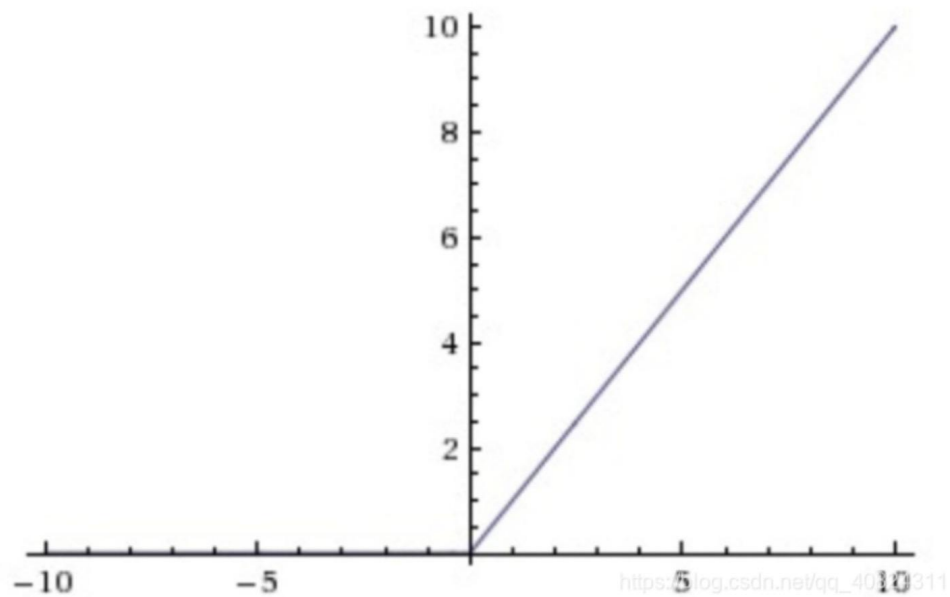


可以看出，相较于Sigmoid函数有所改善，但导数仍小于1，不能避免梯度消失的情况。

7.2.1 代价函数

28

饱和、梯度消失



7.2.1 代价函数

29

□ 常见的激活函数：

- **ReLU函数**：深度神经网络中较为常用的激活函数。当 $x > 0$ 时，导数为1，缓解了梯度爆炸和梯度消失问题。然而当 $x < 0$ 时，ReLU在反向传播过程中梯度为0，导致神经元不能被激活。

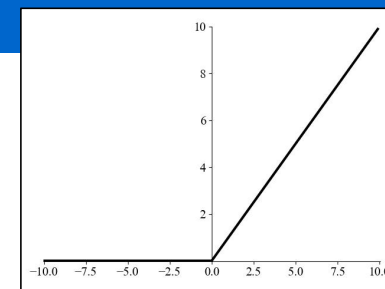
$$\text{ReLU}(x) = \begin{cases} x, & x \geq 0 \\ 0, & x < 0 \end{cases}$$

- **Leaky ReLU函数**：是基于ReLU函数的改进版本，对于 $x < 0$ ，Leaky ReLU具有一个很小的梯度 γ ，使得神经元可以被激活。一般情况下， γ 采用一个很小的常数，如0.01。

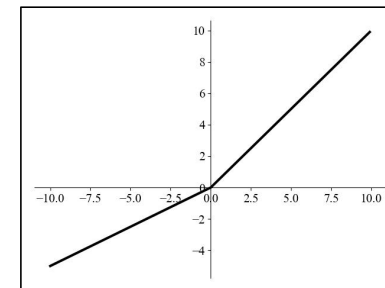
$$\text{LeakyReLU}(x) = \begin{cases} x, & x \geq 0 \\ \gamma x, & x < 0 \end{cases}$$

- **Swish函数**：是一种自门控激活函数，采用Sigmoid函数作为一种门控机制。当 x 较大时， $\sigma(x)$ 趋向于1，Swish函数结果近似于 x 本身，此时Swish函数近似为ReLU函数；当 x 较小时， $\sigma(x)$ 趋向于0，Swish函数结果近似于0。由于Swish激活函数的函数较为复杂，在训练过程中较为耗时。

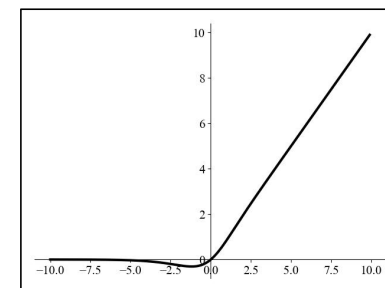
$$\text{swish}(x) = x\sigma(x)$$



(d) ReLU函数



(e) Leaky ReLU函数



(f) Swish函数

7.2.2 输出单元

30

一种简单的输出单元是基于仿射变换的输出单元，仿射变换不具有非线性。这些单元往往被直接称为线性单元。

给定特征 $\mathbf{h}$ ，线性输出单元层产生一个向量 $\hat{\mathbf{y}} = \mathbf{W}^\top \mathbf{h} + \mathbf{b}$ 。

线性输出层经常被用来产生条件高斯分布的均值：

$$p(\mathbf{y} \mid \mathbf{x}) = \mathcal{N}(\mathbf{y}; \hat{\mathbf{y}}, \mathbf{I}).$$

最大化其对数似然此时等价于最小化均方误差。

回归问题求解

7.2.2 输出单元

31

许多任务需要预测二值型变量 y 的值。具有两个类的分类问题可以归结为这种形式。

此时最大似然的方法是定义 y 在 $\mathbf{x}$ 条件下的 Bernoulli 分布。

Bernoulli 分布仅需单个参数来定义。神经网络只需要预测 $P(y = 1 \mid \mathbf{x})$ 即可。为了使这个数是有效的概率，它必须处在区间 $[0, 1]$ 中。

二分类问题求解

7.2.2 输出单元

32

为满足该约束条件需要一些细致的设计工作。假设我们打算使用线性单元，并且通过阈值来限制它成为一个有效的概率：

$$P(y = 1 \mid \mathbf{x}) = \max \{0, \min\{1, \mathbf{w}^\top \mathbf{h} + b\}\}.$$

这的确定义了一个有效的条件概率分布，但我们无法使用梯度下降来高效地训练它。当 $\mathbf{w}^\top \mathbf{h} + b$ 处于单位区间外时，模型的输出对其参数的梯度都将为 **0**。梯度为 **0** 通常是有问题的，因为学习算法对于如何改善相应的参数不再具有指导意义。

二分类问题求解

7.2.2 输出单元

33

sigmoid 输出单元定义为

$$\hat{y} = \sigma(\mathbf{w}^\top \mathbf{h} + b),$$

我们可以认为 sigmoid 输出单元具有两个部分。首先，它使用一个线性层来计算 $z = \mathbf{w}^\top \mathbf{h} + b$ 。接着，它使用 sigmoid 激活函数将 z 转化成概率。

二分类问题求解

7.2.2 输出单元

34

为什么使用最大似然训练Sigmoid:

当我们使用其他的损失函数，例如均方误差之类的，损失函数会在 $\sigma(z)$ 饱和时饱和。sigmoid 激活函数在 z 取非常小的负值时会饱和到 0，当 z 取非常大的正值时会饱和到 1。这种情况一旦发生，梯度会变得非常小以至于不能用来学习，无论此时模型给出的是正确还是错误的答案。因此，最大似然几乎总是训练 sigmoid 输出单元的优选方法。

梯度消失导致无法学习

7.2.2 输出单元

35

任何时候当我们想要表示一个具有 n 个可能取值的离散型随机变量的分布时，我们都可以使用 softmax 函数。它可以看作是 sigmoid 函数的扩展，其中 sigmoid 函数用来表示二值型变量的分布。

$$\text{softmax}(\mathbf{z})_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)}.$$

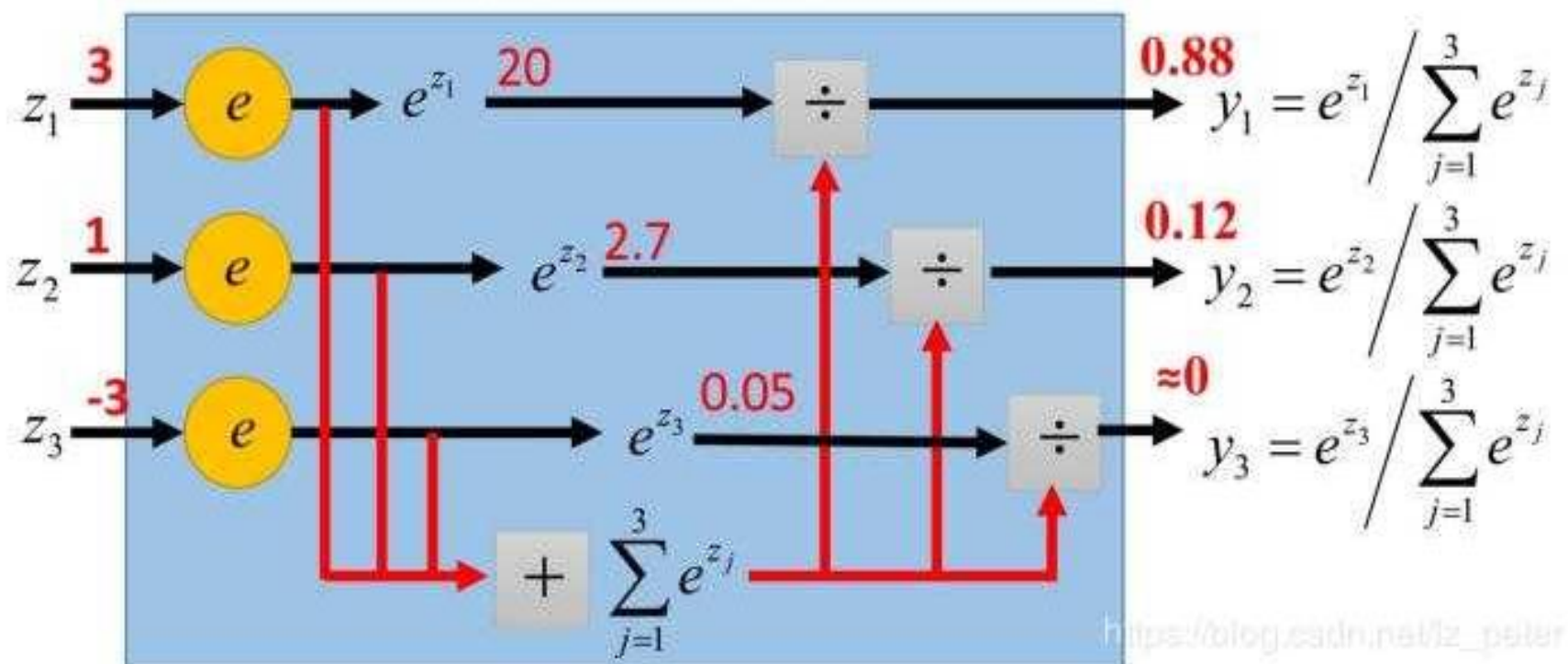
$$z_i = \log \hat{P}(y = i \mid \mathbf{x})$$

多分类问题求解

7.2.2 输出单元

36

$$\text{softmax}(\mathbf{z})_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)}.$$



https://blog.csdn.net/iz_peter

7.2.2 输出单元

37

```
def sigmoid(x):  
    return 1 / (1 + np.exp(-x))
```

```
def sigmoid_derivative(x):  
    sig = sigmoid(x)  
    return sig * (1 - sig)
```

$$\sigma(x) = \frac{1}{1 + \exp(-x)}.$$

$$\text{softmax}(\mathbf{z})_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)}.$$

```
def softmax(x):  
    exps = np.exp(x)  
    sum_exps = np.sum(exps, axis=1, keepdims=True)  
    return exps / sum_exps
```

```
def softmax_derivative(x):  
    s = softmax(x)  
    return s * (1 - s)
```

7.2.2 输出单元

38

一般的，如果我们定义了一个条件分布 $p(\mathbf{y} | \mathbf{x}; \boldsymbol{\theta})$ ，最大似然原则建议我们使用 $-\log p(\mathbf{y} | \mathbf{x}; \boldsymbol{\theta})$ 作为代价函数。

假设分布	输出单元	代价函数	适用问题
高斯分布	线性单元	均方误差损失函数	回归
伯努利分布	Sigmiod	交叉熵损失函数	二分类
多项式分布	Softmax	多类别交叉熵损失函数	多分类

7.3 隐藏单元

39

整流线性单元是隐藏单元极好的默认选择。许多其他类型的隐藏单元也是可用的。决定何时使用哪种类型的隐藏单元是困难的事（尽管整流线性单元通常是一个可接受的选择）。

除非另有说明，大多数的隐藏单元都可以描述为接受输入向量 $\mathbf{x}$ ，计算仿射变换 $\mathbf{z} = \mathbf{W}^\top \mathbf{x} + \mathbf{b}$ ，然后使用一个逐元素的非线性函数 $g(\mathbf{z})$ 。大多数隐藏单元的区别仅仅在于激活函数 $g(\mathbf{z})$ 的形式。

7.3.1 ReLU及其扩展

40

整流线性单元使用激活函数 $g(z) = \max\{0, z\}$ 。

整流线性单元易于优化，因为它们和线性单元非常类似。线性单元和整流线性单元的唯一区别在于整流线性单元在其一半的定义域上输出为零。这使得只要整流线性单元处于激活状态，它的导数都能保持较大。它的梯度不仅大而且一致。整流操作的二阶导数几乎处处为 0，并且在整流线性单元处于激活状态时，它的一阶导数处处为 1。这意味着相比于引入二阶效应的激活函数来说，它的梯度方向对于学习来说更加有用。

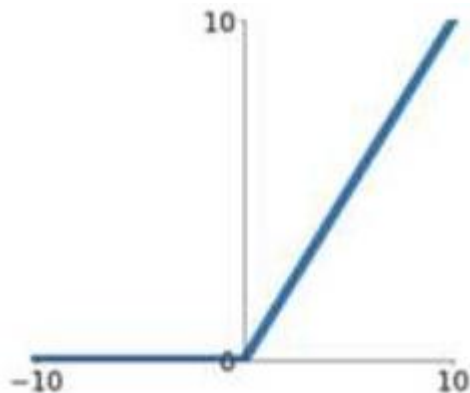
7.3.1 ReLU及其扩展

41

整流线性单元通常作用于仿射变换之上：

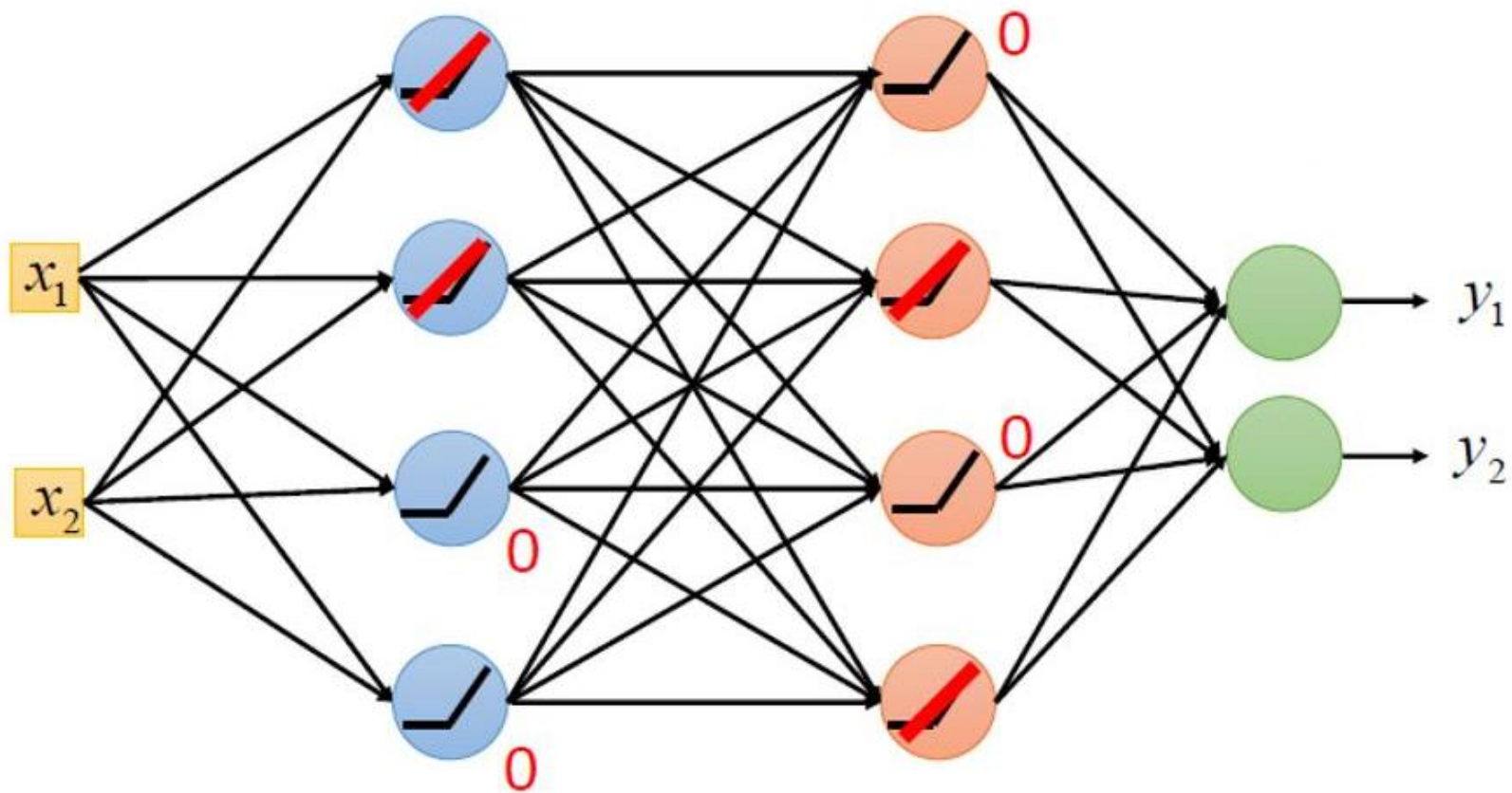
$$h = g(\mathbf{W}^\top \mathbf{x} + \mathbf{b}).$$

整流线性单元的一个缺陷是它们不能通过基于梯度的方法学习那些使它们激活为零的样本。整流线性单元的各种扩展保证了它们能在各个位置都接收到梯度。



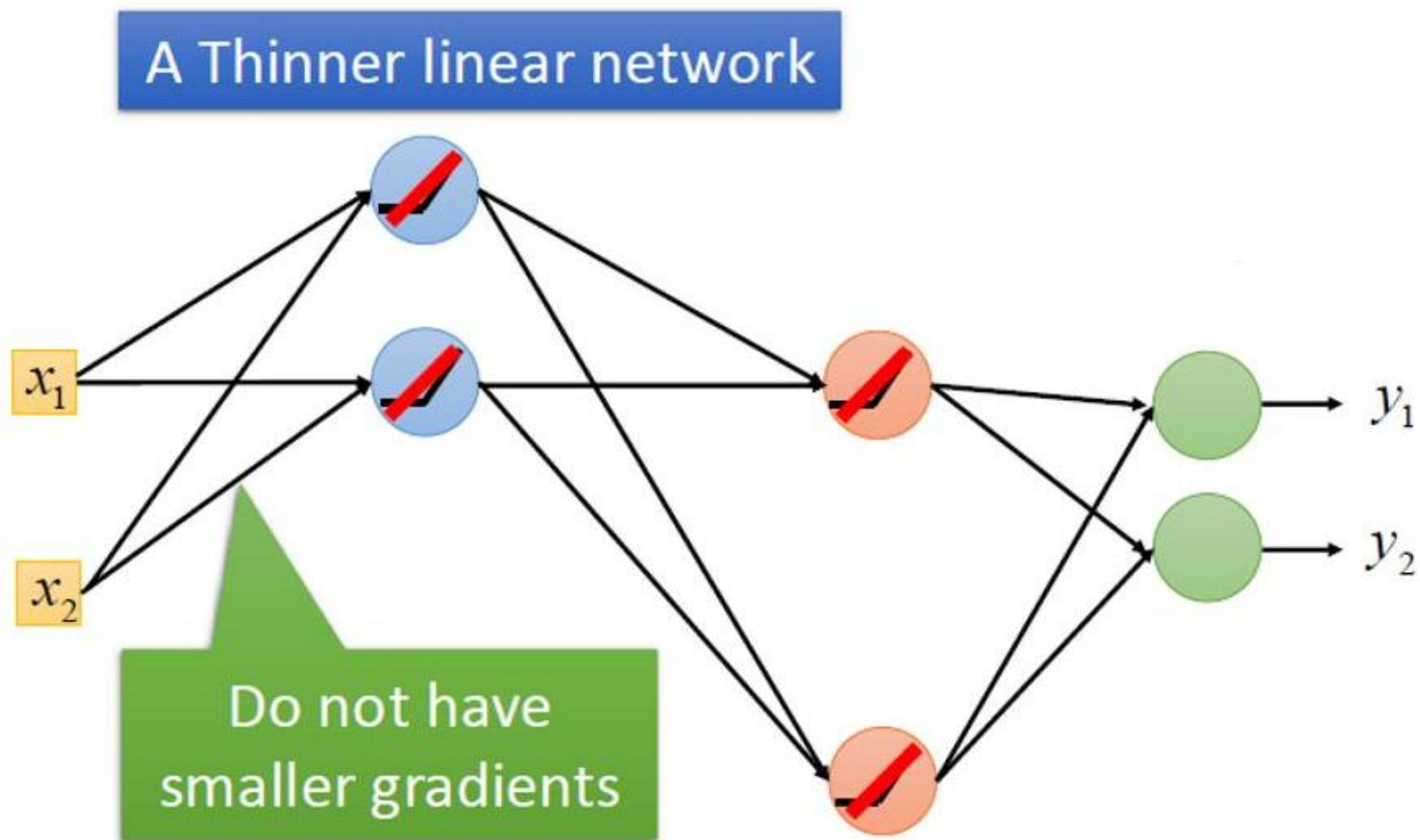
7.3.1 ReLU及其扩展

42



7.3.1 ReLU及其扩展

43

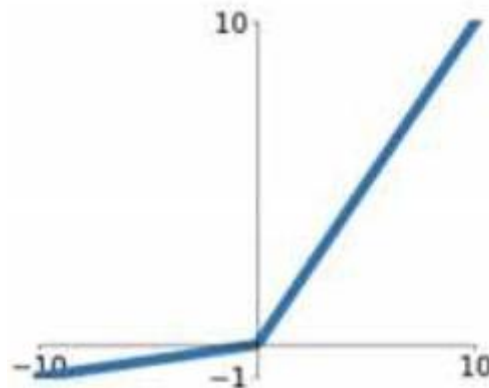


7.3.1 ReLU及其扩展

44

整流线性单元的三个扩展基于当 $z_i < 0$ 时使用一个非零的斜率 α_i : $h_i = g(\mathbf{z}, \boldsymbol{\alpha})_i = \max(0, z_i) + \alpha_i \min(0, z_i)$ 。绝对值整流 (absolute value rectification) 固定 $\alpha_i = -1$ 来得到 $g(z) = |z|$ 。它用于图像中的对象识别 (Jarrett *et al.*, 2009a), 其中寻找在输入照明极性反转下不变的特征是有意义的。整流线性单元的其他扩展比这应用地更广泛。渗漏整流线性单元 (Leaky ReLU) (Maas *et al.*, 2013) 将 α_i 固定成一个类似 0.01 的小值, 参数化整流线性单元 (parametric ReLU) 或者 PReLU 将 α_i 作为学习的参数 (He *et al.*, 2015)。

$$g(\mathbf{z}, \boldsymbol{\alpha})_i = \max(0, z_i) + \alpha_i \min(0, z_i)$$



7.3.2 sigmoid和tanh

45

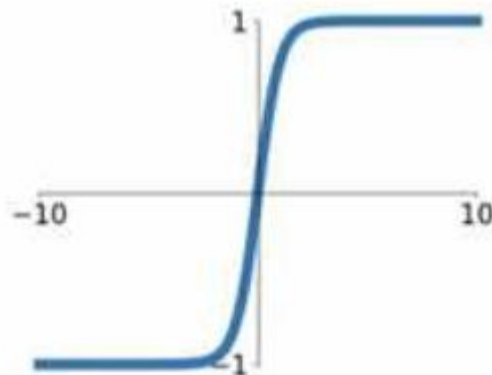
在引入整流线性单元之前，大多数神经网络使用 logistic sigmoid 激活函数

$$g(z) = \sigma(z)$$

或者是双曲正切激活函数

$$g(z) = \tanh(z).$$

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{e^{2x} - 1}{e^{2x} + 1}$$



7.3.2 sigmoid和tanh

46

风中摇曳的小萝卜 bilibili

5分钟了解深度学习算法



激活函数

7.4 架构设计

47

大多数神经网络被组织成称为层的单元组。大多数神经网络架构将这些层布置成链式结构，其中每一层都是前一层的函数。在这种结构中，第一层由下式给出：

$$\mathbf{h}^{(1)} = g^{(1)} \left(\mathbf{W}^{(1)\top} \mathbf{x} + \mathbf{b}^{(1)} \right);$$

第二层由

$$\mathbf{h}^{(2)} = g^{(2)} \left(\mathbf{W}^{(2)\top} \mathbf{h}^{(1)} + \mathbf{b}^{(2)} \right);$$

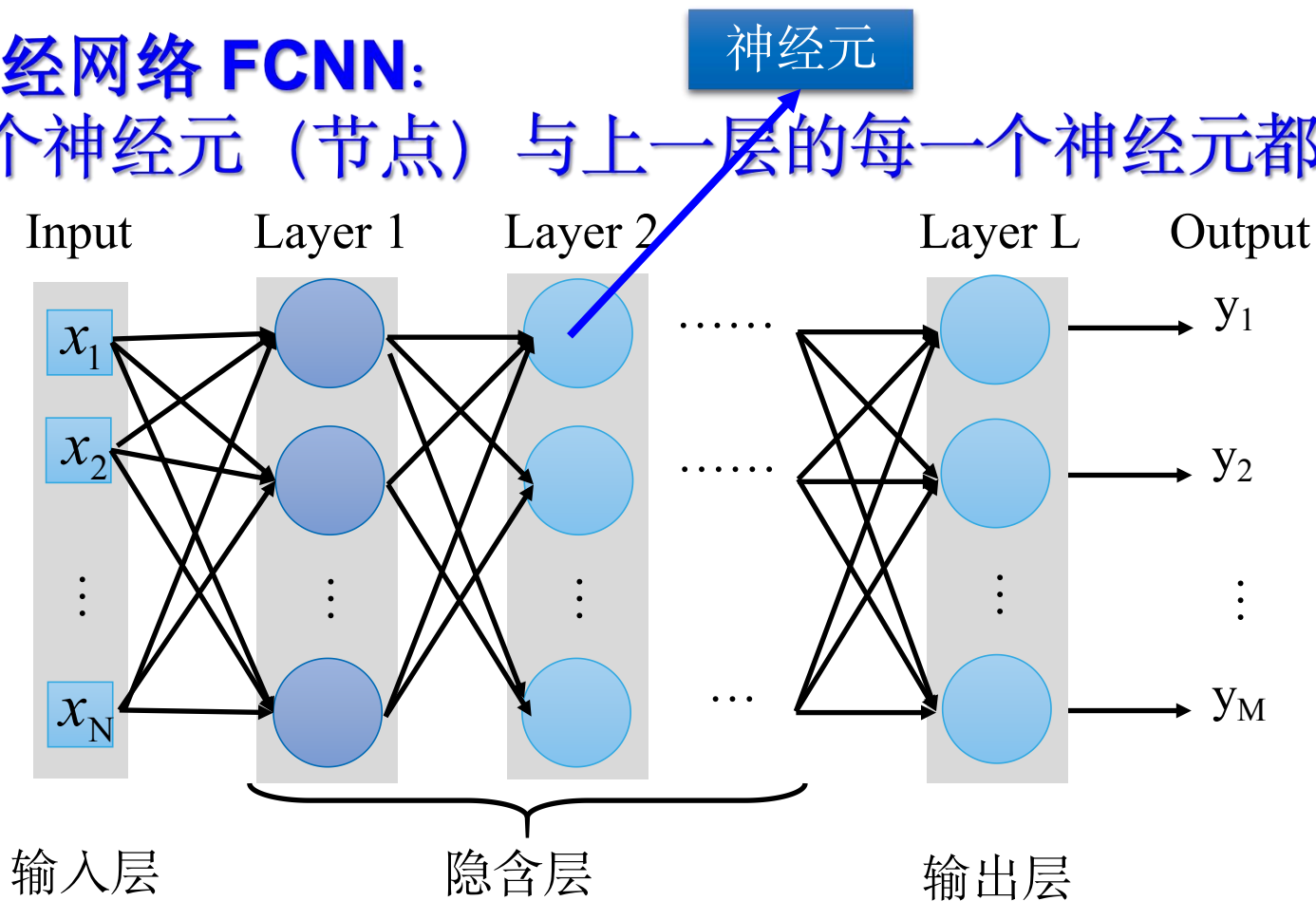
给出，以此类推。

7.4 架构设计

48

□ 全连接神经网络 FCNN:

当前层每一个神经元（节点）与上一层的每一个神经元都有连接。

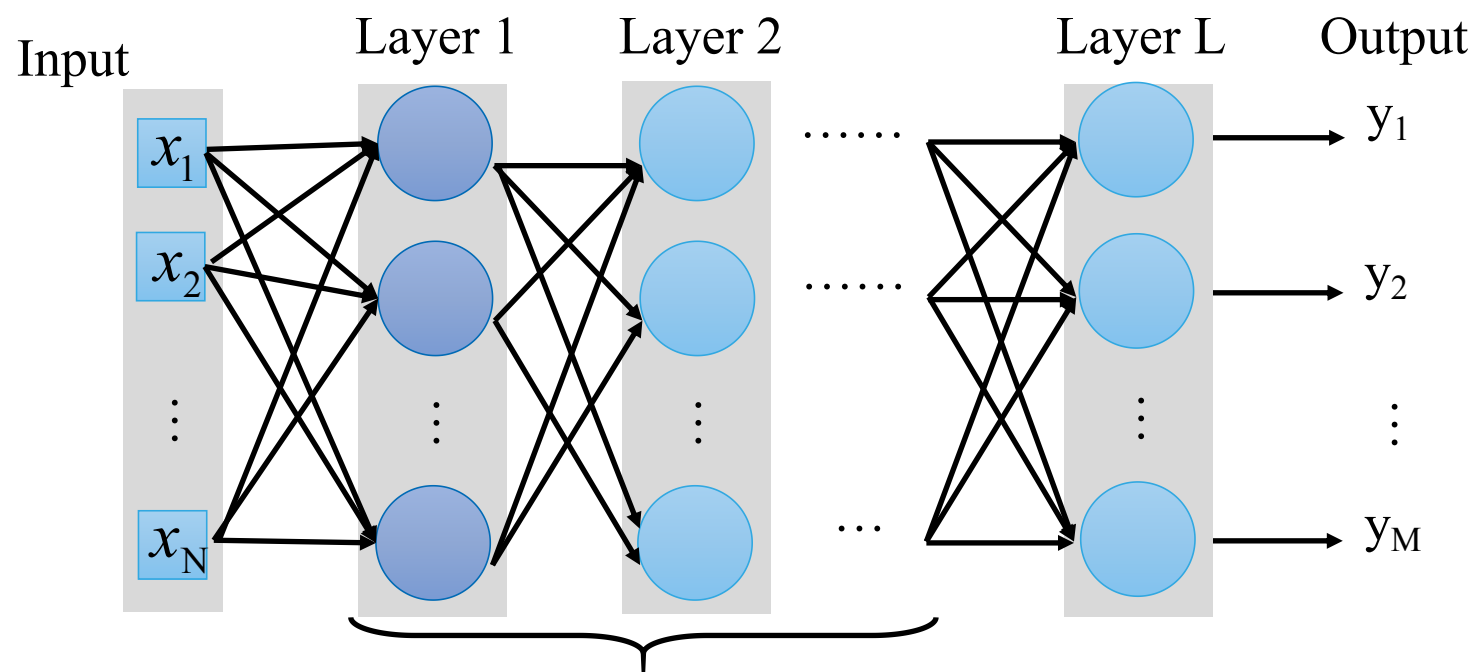


Deep means many hidden layers

7.4 架构设计

49

□ 数据流传播方向：前向传播



数据流方向：前向传播 (forward)

网络
输出

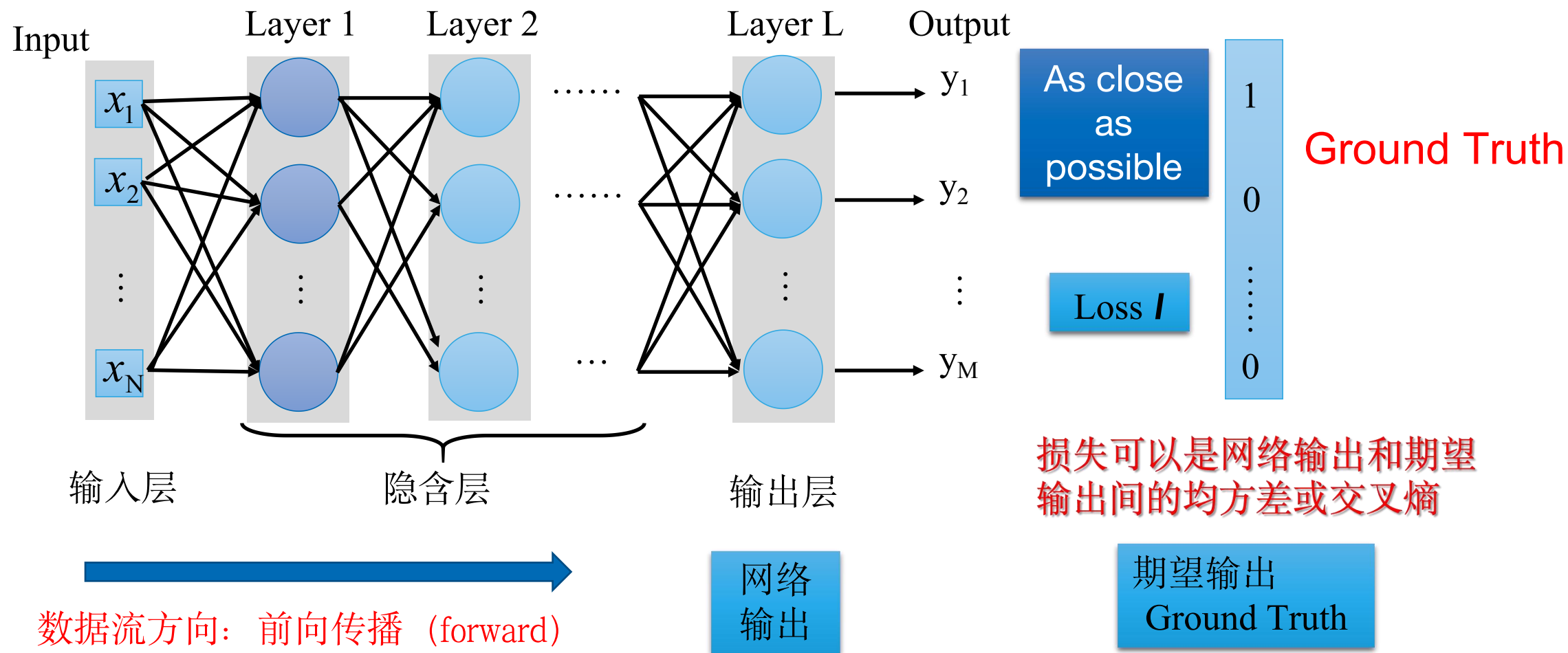
$$h^{(1)} = g^{(1)} \left(W^{(1)\top} x + b^{(1)} \right)$$

$$h^{(2)} = g^{(2)} \left(W^{(2)\top} h^{(1)} + b^{(2)} \right)$$

7.4 架构设计

50

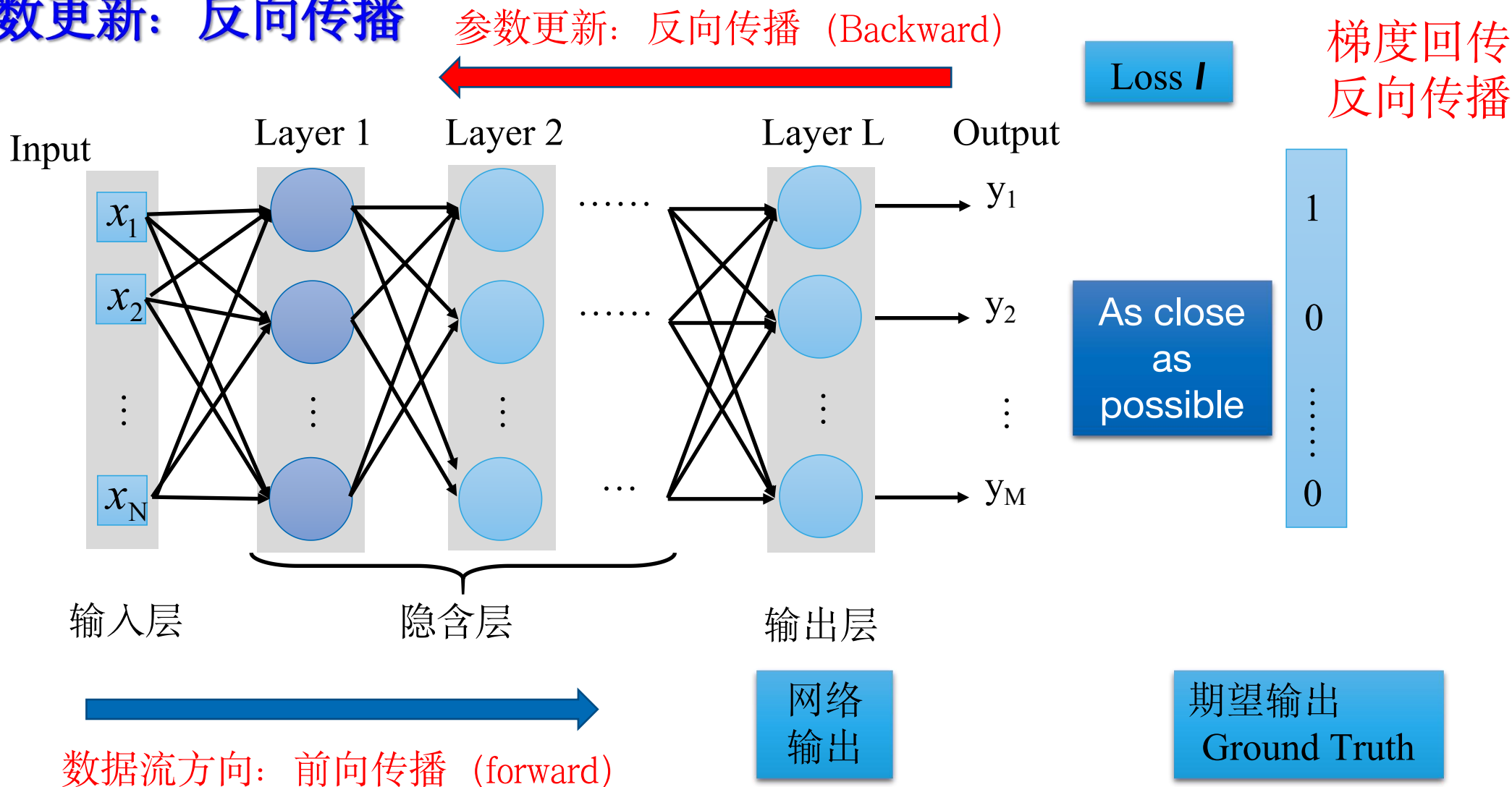
□ 损失函数:



7.4 架构设计

51

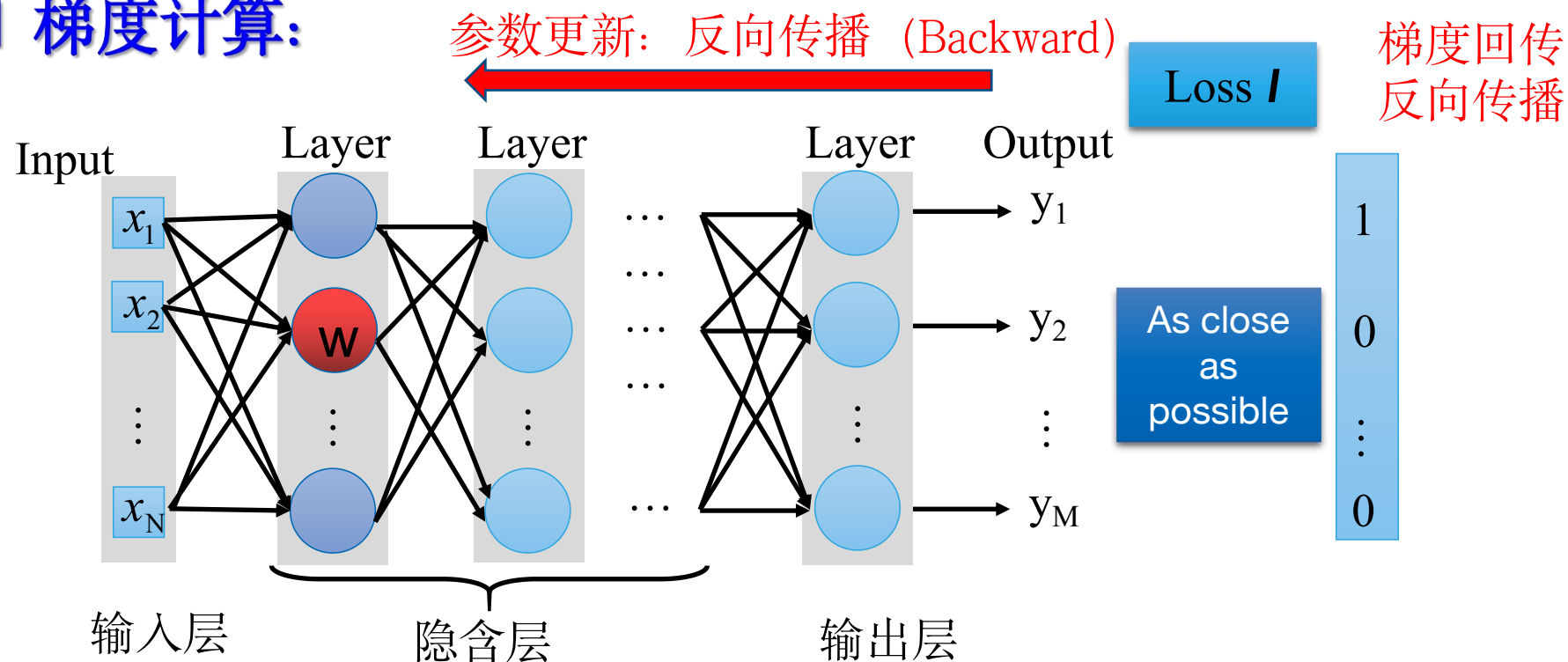
参数更新: 反向传播



7.4 架构设计

52

□ 梯度计算:

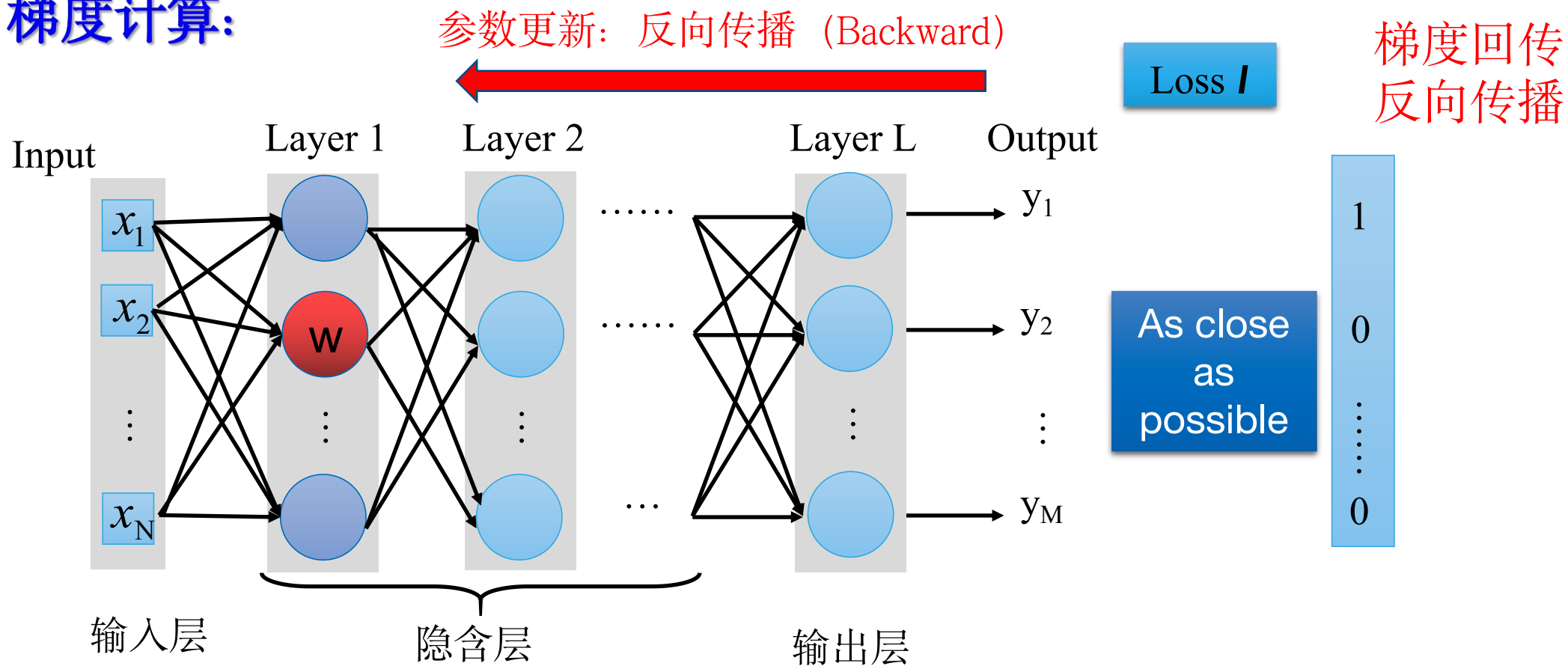


- 前向传播: 输入数据通过网络, 得到预测输出
- 计算损失: 比较预测输出 y' 和真实标签 y , 计算差 (Loss)
- 反向传播: 将损失从输出层反向传播回输入层

7.4 架构设计

53

□ 梯度计算:



调整参数 $\rightarrow$ 参数更新 $\rightarrow$ 求梯度

如何求loss对红色神经元的梯度?



7.5 反向传播

55

当我们使用前馈神经网络接收输入 x 并产生输出 $\hat{y}$ 时，信息通过网络向前流动。输入 x 提供初始信息，然后传播到每一层的隐藏单元，最终产生输出 $\hat{y}$ 。这称之为 **前向传播**（forward propagation）。在训练过程中，前向传播可以持续向前直到它产生一个标量代价函数 $J(\theta)$ 。**反向传播**（back propagation）算法（Rumelhart *et al.*, 1986c），经常简称为**backprop**，允许来自代价函数的信息通过网络向后流动，以便计算梯度。

7.5.1 计算图

56

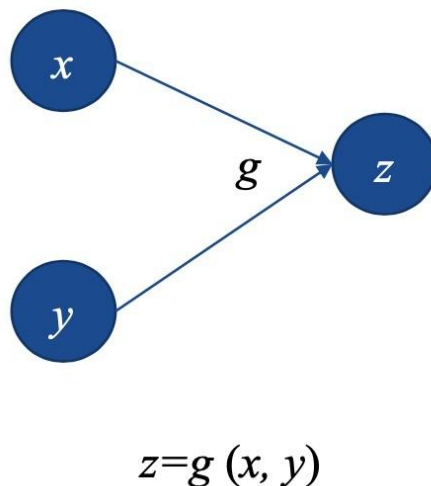
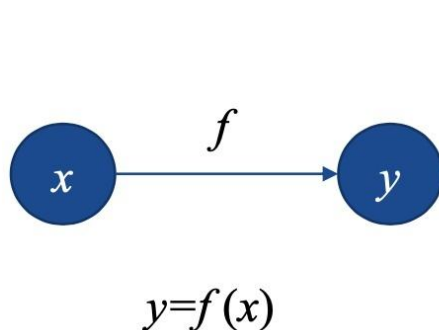
□ 计算图(Computational Graph)

– 描述计算结构的一种图

□ 计算图的元素

– 节点(node): 表示变量, 可以是标量、矢量、张量等

– 边(edge): 表示操作(函数)



7.5.1 计算图

57

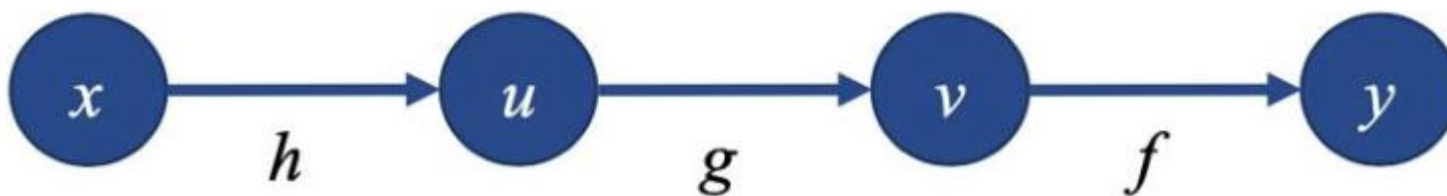
$y = xe^{x^2}$ 的计算图?

$$y = f(g(h(x)))$$

$$u = h(x)$$

$$v = g(u)$$

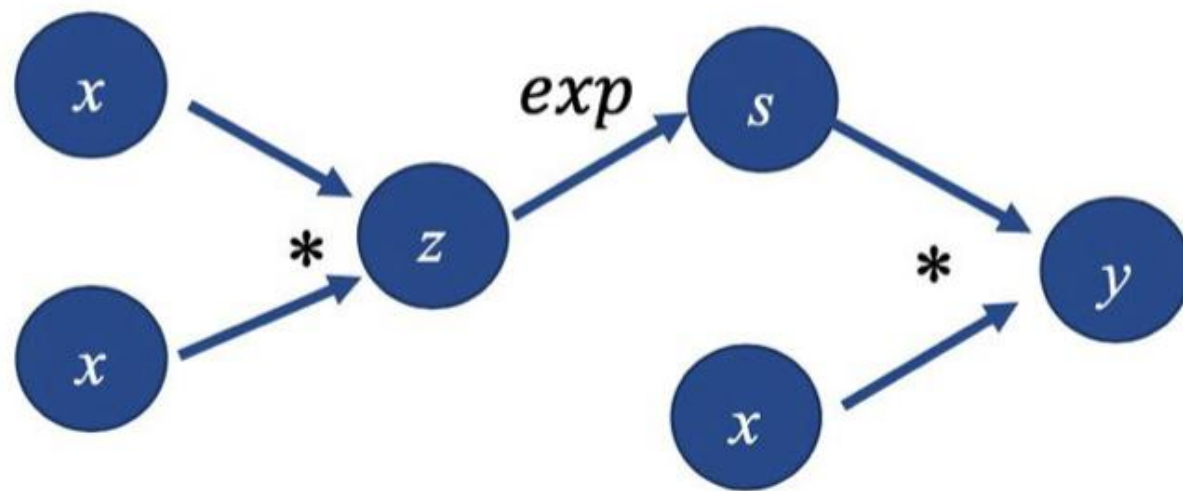
$$y = f(v)$$



7.5.1 计算图

58

$$y = xe^{x^2}$$

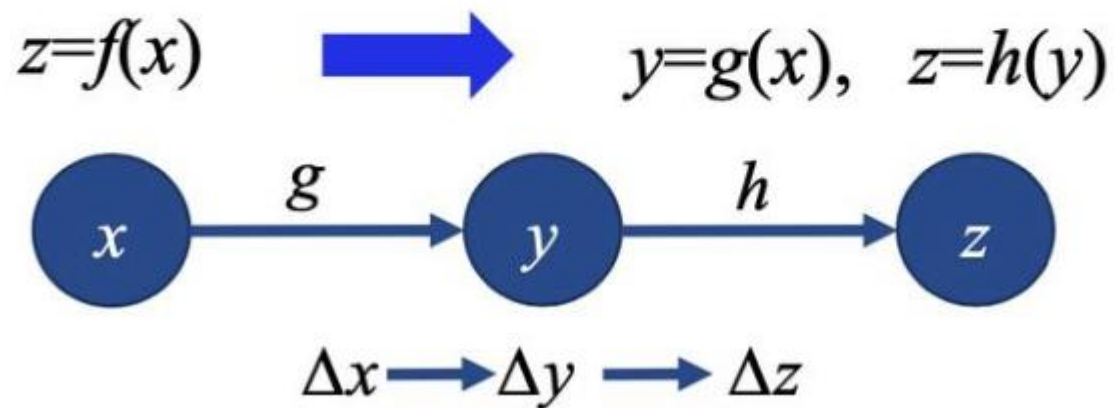


7.5.1 计算图

59

□ 链式法则

– Case 1:



$$\frac{dz}{dx} = \frac{dz}{dy} \frac{dy}{dx}$$

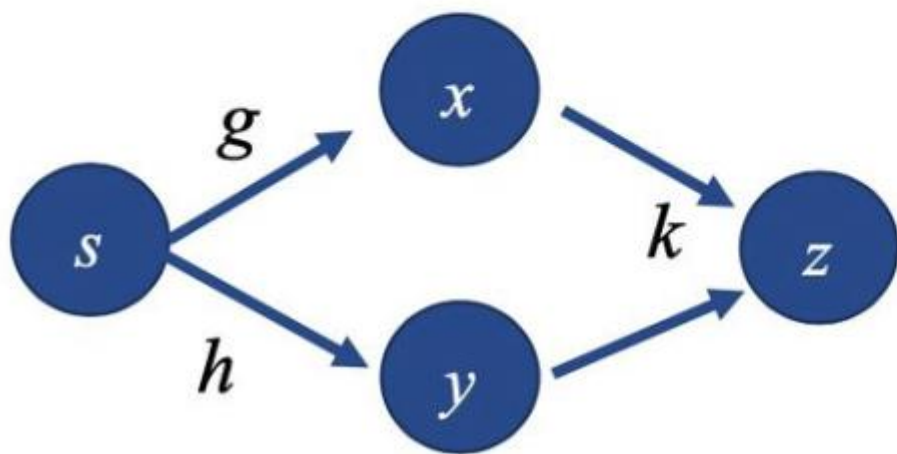
7.5.1 计算图

60

□ 链式法则

– Case 2:

$$z=f(s) \quad \longrightarrow \quad x=g(s), \quad y=h(s), \quad z=k(x,y)$$



A differential graph with three nodes: Δs , Δx , and Δy . Node Δs is on the left. Two arrows originate from Δs : one pointing to node Δx and one pointing to node Δy . From node Δx , an arrow points to node Δz . From node Δy , an arrow points to node Δz .

$$\frac{dz}{ds} = \frac{\partial z}{\partial x} \times \frac{\partial x}{\partial s} + \frac{\partial z}{\partial y} \times \frac{\partial y}{\partial s}$$

算法 6.3 典型深度神经网络中的前向传播和代价函数的计算。损失函数 $L(\hat{\mathbf{y}}, \mathbf{y})$ 取决于输出 $\hat{\mathbf{y}}$ 和目标 $\mathbf{y}$ (参考第 6.2.1.1 节中损失函数的示例)。为了获得总代价 J , 损失函数可以加上正则项 $\Omega(\theta)$, 其中 θ 包含所有参数 (权重和偏置)。算法 6.4 说明了如何计算 J 关于参数 $\mathbf{W}$ 和 $\mathbf{b}$ 的梯度。为简单起见, 该演示仅使用单个输入样本 $\mathbf{x}$ 。实际应用应该使用小批量。请参考第 6.5.7 节以获得更加真实的演示。

Require: 网络深度, l

Require: $\mathbf{W}^{(i)}, i \in \{1, \dots, l\}$, 模型的权重矩阵

Require: $\mathbf{b}^{(i)}, i \in \{1, \dots, l\}$, 模型的偏置参数

Require: $\mathbf{x}$, 程序的输入

Require: $\mathbf{y}$, 目标输出

$$\mathbf{h}^{(0)} = \mathbf{x}$$

for $k = 1, \dots, l$ **do**

$$\mathbf{a}^{(k)} = \mathbf{b}^{(k)} + \mathbf{W}^{(k)} \mathbf{h}^{(k-1)}$$

$$\mathbf{h}^{(k)} = f(\mathbf{a}^{(k)})$$

end for

$$\hat{\mathbf{y}} = \mathbf{h}^{(l)}$$

$$J = L(\hat{\mathbf{y}}, \mathbf{y}) + \lambda \Omega(\theta)$$

前向传播

算法 6.4 深度神经网络中算法 6.3 的反向计算，它不止使用了输入 x 和目标 y 。该计算对于每一层 k 都产生了对激活 $a^{(k)}$ 的梯度，从输出层开始向后计算一直到第一个隐藏层。这些梯度可以看作是对每层的输出应如何调整以减小误差的指导，根据这些梯度可以获得对每层参数的梯度。权重和偏置上的梯度可以立即用作随机梯度更新的一部分（梯度算出后即可执行更新），或者与其他基于梯度的优化方法一起使用。

在前向计算完成后，计算顶层的梯度：

$$g \leftarrow \nabla_{\hat{y}} J = \nabla_{\hat{y}} L(\hat{y}, y)$$

for $k = l, l-1, \dots, 1$ **do**

将关于层输出的梯度转换为非线性激活输入前的梯度（如果 f 是逐元素的，则逐元素地相乘）：

$$g \leftarrow \nabla_{a^{(k)}} J = g \odot f'(a^{(k)})$$

计算关于权重和偏置的梯度（如果需要的话，还要包括正则项）：

$$\nabla_{b^{(k)}} J = g + \lambda \nabla_{b^{(k)}} \Omega(\theta)$$

$$\nabla_{W^{(k)}} J = g h^{(k-1)\top} + \lambda \nabla_{W^{(k)}} \Omega(\theta)$$

关于下一更低层的隐藏层传播梯度：

$$g \leftarrow \nabla_{h^{(k-1)}} J = W^{(k)\top} g$$

end for

反向传播

$$a^{(k)} = b^{(k)} + W^{(k)} h^{(k-1)}$$

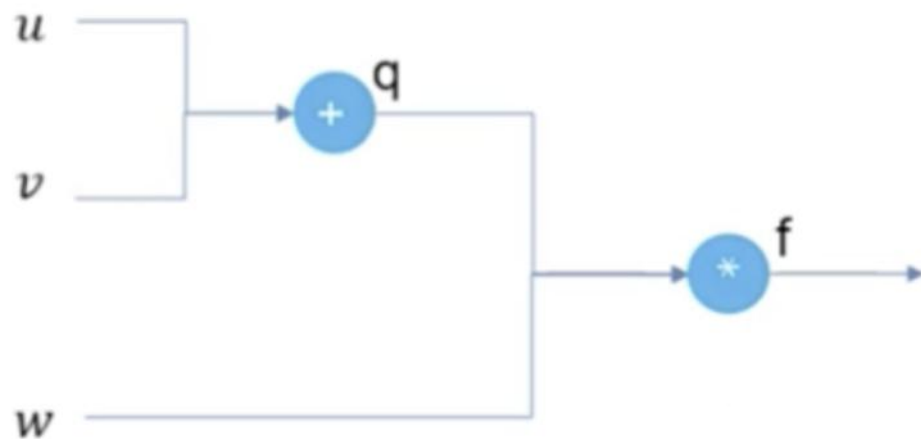
练习题

63

例：假设复合表达式为 $f(u, v, w) = (u + v)w$ 。令 $q = u + v$ ，则 $f = qw$ ，

则有 $\frac{\partial f}{\partial q} = w$, $\frac{\partial f}{\partial w} = q$, $\frac{\partial q}{\partial u} = 1$, $\frac{\partial q}{\partial v} = 1$,

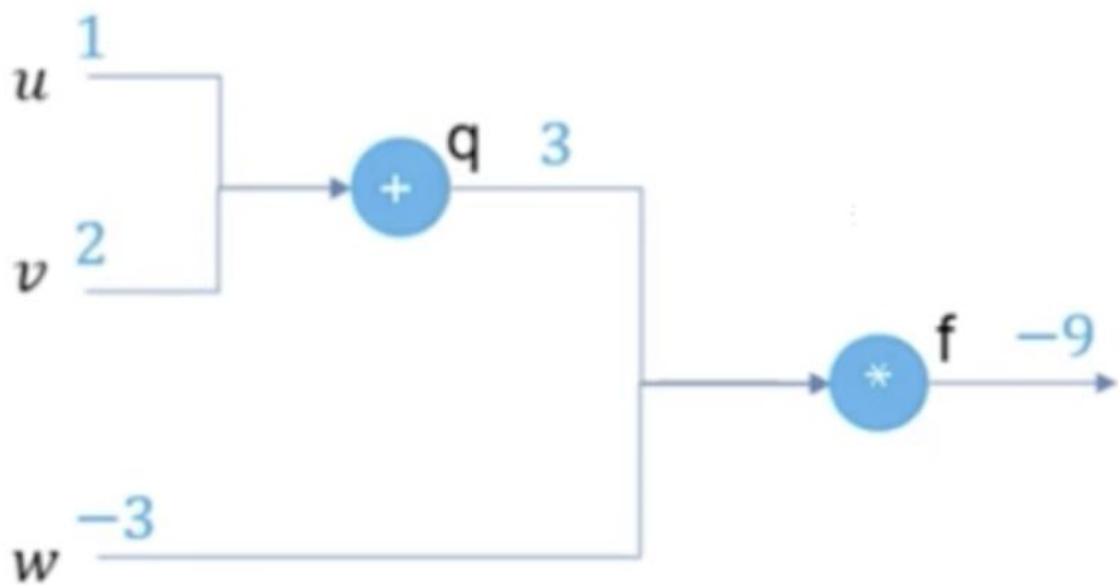
使用链规则，则有 $\frac{\partial f}{\partial u} = \frac{\partial f}{\partial q} \cdot \frac{\partial q}{\partial u} = w * 1$, $\frac{\partial f}{\partial v} = \frac{\partial f}{\partial q} \cdot \frac{\partial q}{\partial v} = w * 1$, $\frac{\partial f}{\partial w} = q$



练习题

64

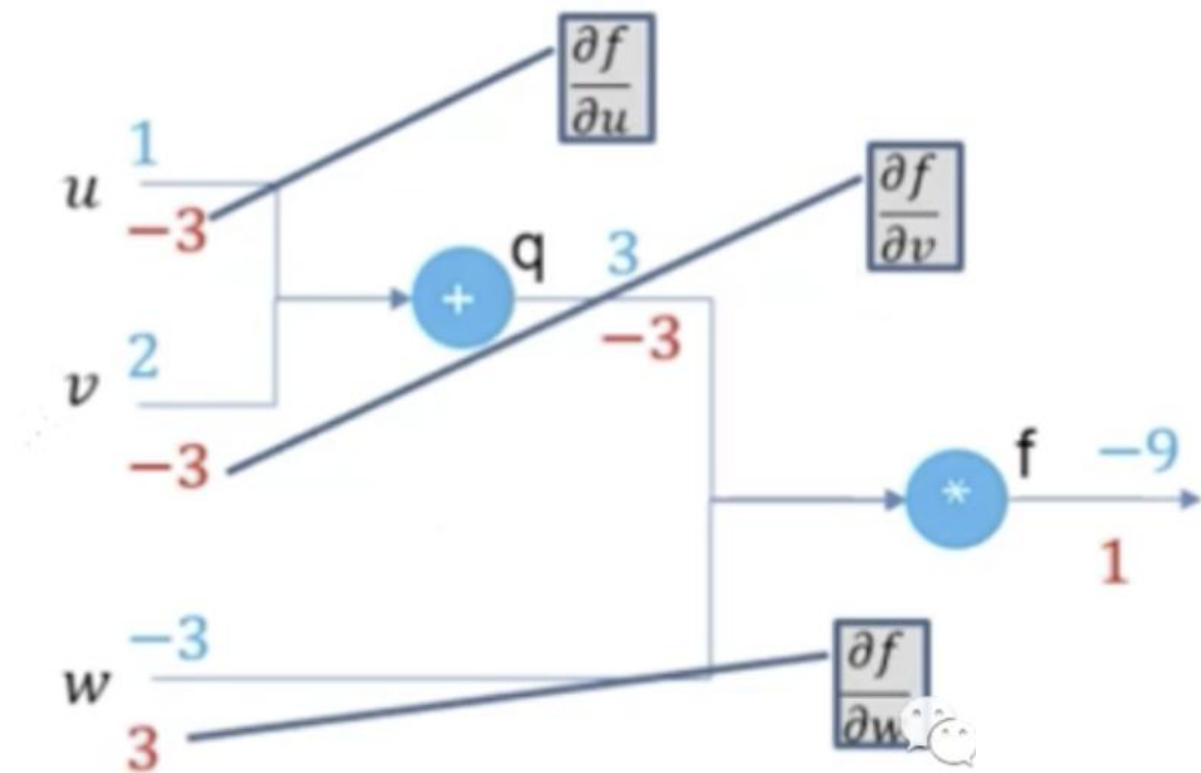
- $u = 1; v = 2; w = -3$
- # 正向传播
- $q = u + v$
- $f = q * w$



练习题

65

- $u = 1; v = 2; w = -3$
- # 正向传播
- $q = u + v$
- $f = q * w$
- # 反向传播
- $dfdq = w$
- $dfdu = 1.0 * dfdq$
- $dfdv = 1.0 * dfdq$
- $dfdw = q$



预训练模型实践或DM实践

预训练实践关键词: Pretrained model、预训练、微调等

DM实践关键词: 数据生成/增强、DDPM、DDIM、GAN等

- 分为6组, 自行组队, 每组4-6人。
- 每组汇报15分钟, 包括演示代码运行, 有5分钟答辩
- 时间节点1: 12月22日~12月29日汇报(第15-16周), 确定题目后登记任务名称和组员姓名, 根据登记顺序汇报
- 时间节点2: 以报告形式于1.9前上交, 有代码者附代码截图和运行结果
- 要求: 各小组任务不可重复, 代码必须是可以顺利运行, ppt演示应包含模型介绍和小组分工。
- 代码跑通并且有结果分析占30%, 汇报占45%, 难度占20%, 专业相关度占5%